Project Tic Tac Toe

Features :

1. 2 players take turns in playing the game (Xs and Os) with desired timer. (Aryan)
2. Coin toss to figure out which player is X and which player is O. (Aryan)
3. Deciding winner through diagonal rows or column checking for 3 in a row. (Ajwad)
4. Deciding draw when the board is full yet no winner. (Ajwad)
5. Checking validity of the moves given by the player. (Amreen)
6. History of moves and scores between the players. (Amreen)

Code Organization by Contributors:

Aryan's Section (Features 1 & 2):

- **Feature 1 - Player turns with timer**: `player_turn`, `switch_player`, `get_player_names`
- **Feature 2 - Coin toss**: `coin_toss`, `title_screen`, `ask_replay`
- **Grid Display**: `display_boards` (moved from Amreen's section)

Ajwad's Section (Features 3 & 4):

- **Feature 3 - Winner checking**: `check_game_over`, `set_winner`
- **Feature 4 - Draw detection**: Part of `check_game_over`, `display_result`

Amreen's Section (Features 5 & 6):

- **Feature 5 - Move validation**: `reset_game`, `record_move`
- **Feature 6 - History and scores**: `display_score`, `display_history`

ARYAN - Player Management & Game Flow Control

Overview: Aryan handled the core player interaction mechanics and game initialization, focusing on creating a smooth user experience with proper turn management and timing systems.

Detailed Analysis of Aryan's Contributions:

1. Title Screen & Initial Setup (`title_screen` procedure)

- Creates the game's first impression with a welcoming interface
- Uses DOS interrupt 21h with function 9 to display formatted strings
- Implements a controlled wait loop that specifically waits for the spacebar press
- Uses keyboard interrupt 16h with function 0 for blocking input
- Strategic use of screen clearing to provide a clean slate for each game session

2. Player Registration System (`get_player_names` procedure)

- Sophisticated input handling using DOS function 0Ah for buffered string input
- Implements proper memory management for player name storage (18-character limit)
- Uses dynamic null-termination by calculating string length and adding '$' terminator
- Integrates timer selection (10s, 20s, 40s) with input validation and retry logic
- Manages memory offsets carefully to store names in separate buffers

3. Coin Toss Mechanism (`coin_toss` procedure)

- Implements pseudo-random number generation using BIOS timer interrupt 1Ah
- Uses bit masking (AND with 1) to create binary randomness

- Provides immediate visual feedback with proper name display
- Includes deliberate delay (interrupt 15h, function 86h) for dramatic effect
- Determines X/O assignments that persist throughout the game

4. Advanced Turn Management (`player_turn` procedure)

- **Real-time Timer System**: Implements a countdown timer with live display updates
- Uses cursor positioning (interrupt 10h, function 2) for in-place timer updates
- Handles both single and double-digit timer display with ASCII conversion
- **Non-blocking Input Checking**: Uses keyboard interrupt 16h, function 1 to check for available input without waiting
- **Input Validation**: Converts ASCII input to numeric values and validates range (1-9)
- **Board State Management**: Checks if positions are already occupied before allowing moves
- **History Integration**: Stores move information for later retrieval
- **Timeout Handling**: Automatically switches players when time expires

5. Player State Switching (`switch_player` procedure)

- Manages the complex relationship between player numbers (1/2) and marks (X/O)
- Correctly assigns marks based on coin toss winner (who gets X)
- Ensures proper alternation between players while maintaining mark consistency

6. Game Board Visualization (`display_boards` procedure)

- **Dual Grid System**: Displays both position reference grid (1-9) and actual game state side-by-side
- **Precise Character Positioning**: Uses individual character output for perfect alignment
- **Dynamic Player Information**: Shows current X/O assignments based on coin toss results
- **Interactive History Access**: Provides option to view move history during gameplay

- **Grid Formatting**: Creates professional-looking tic-tac-toe grids with proper spacing and separators

---

AJWAD - Game Logic & Victory Conditions

Overview: Ajwad implemented the core game intelligence, handling all win/draw detection algorithms and game state management that determines when games end.

Detailed Analysis of Ajwad's Contributions:

1. Comprehensive Win Detection (`check_game_over` procedure)

- **Row Checking Algorithm**: Systematically checks all three horizontal rows using loop iteration
- **Column Checking Logic**: Implements vertical win detection with mathematical indexing (positions 0,3,6 / 1,4,7 / 2,5,8)
- **Diagonal Detection**: Handles both main diagonal (0,4,8) and anti-diagonal (2,4,6) patterns
- **Efficient Early Termination**: Stops checking once a winner is found to optimize performance
- **Empty Space Validation**: Ensures only occupied positions contribute to win conditions

2. Winner State Management (`set_winner` procedure)

- Cleanly sets game termination flags when victory conditions are met
- Correctly identifies the winning player based on the current player who just moved
- Updates global game state variables for result processing

3. Draw Detection Logic (within `check_game_over`)

- **Board Fullness Algorithm**: Iterates through all 9 positions to detect completely filled board
- **Draw vs Win Priority**: Ensures win conditions are checked before draw conditions

- **State Differentiation**: Properly distinguishes between draw (winner_player = 0) and win states

4. Comprehensive Result Display (`display_result` procedure)

- **Victory Celebration**: Creates visual celebration with asterisks and formatting for winners
- **Score Management**: Automatically increments winner's score counter
- **Draw Handling**: Provides appropriate messaging for tied games
- **Player Name Integration**: Dynamically displays winner names from stored player data
- **Interactive History Access**: Allows players to review game history after completion
- **Screen Management**: Coordinates with display systems to show final game state

5. Replay System (`ask_replay` procedure)

- **Input Validation Loop**: Continuously checks for valid R/r (replay) or Q/q (quit) inputs
- **Return Value System**: Uses register AL to communicate user choice back to main game loop
- **Graceful Exit**: Provides thank-you message before program termination
- **Case Insensitive Input**: Handles both uppercase and lowercase user inputs

---

AMREEN - Data Management & User Experience

Overview: Amreen focused on data integrity, move validation, historical record keeping, and utility functions that support the entire game infrastructure.

Detailed Analysis of Amreen's Contributions:

1. Game State Initialization (`reset_game` procedure)

- **Board Reset Logic**: Systematically clears all 9 board positions to spaces

- **State Variable Management**: Properly initializes all game variables for new rounds
- **History System Reset**: Clears move history arrays while preserving player scores
- **Player Continuity**: Maintains toss winner information across game rounds
- **Timer Synchronization**: Ensures timer settings persist between games

2. Move History System (`record_move` procedure)

- **Multi-dimensional Tracking**: Records player identity, mark used, and position chosen
- **Dynamic Array Management**: Uses history_count as index for sequential storage
- **Data Type Conversion**: Converts numeric positions to ASCII for display purposes
- **Chronological Ordering**: Maintains move sequence for replay analysis

3. Score Tracking & Display (`display_score` procedure)

- **Professional Formatting**: Creates structured score display with headers and separators
- **Dynamic Name Display**: Shows actual player names rather than generic labels
- **Score Persistence**: Maintains running totals across multiple game rounds
- **ASCII Conversion**: Properly converts numeric scores to displayable characters

4. Historical Data Visualization (`display_history` procedure)

- **Comprehensive Move Replay**: Shows complete chronological game progression
- **Player Attribution**: Links each move to specific player names and marks
- **Move Numbering**: Provides sequential move numbering for easy reference
- **Empty State Handling**: Gracefully handles cases where no moves have been made yet
- **Interactive Display**: Waits for user acknowledgment before continuing

5. Utility Infrastructure (Shared Utilities)

- **Screen Management (`clear_screen`)**: Provides consistent screen clearing using BIOS video interrupt
- **Display Coordination**: Ensures clean visual transitions between game states
- **Cross-procedure Support**: Enables other contributors' features through foundational utilities

Data Structure Management

Amreen designed and managed several critical data structures:

- **History Arrays**: `history_players`, `history_marks`, `history_positions` for comprehensive move tracking
- **Score Variables**: `p1_score`, `p2_score` for persistent scoring across games
- **Player Information**: `p1_name`, `p2_name` buffers with proper memory allocation
- **Game State Flags**: `game_over`, `current_player`, `history_count` for state management

Input Validation Architecture

- Integrated validation logic within the turn system to ensure data integrity
- Prevents invalid moves through position availability checking
- Maintains consistent data flow between user input and internal game state

---

Integration Analysis

The three contributors created a cohesive system where:

- **Aryan's player management** feeds into **Ajwad's game logic**
- **Ajwad's win detection** triggers **Amreen's score tracking**
- **Amreen's history system** records **Aryan's turn management**
- All three systems work together through carefully managed shared data structures and procedure calls

This modular approach demonstrates excellent software engineering principles while maintaining tight integration between all game components.

Code :

```
.stack 100h
.data
    ; Game board and display
    board db ' ',' ',' ',' ',' ',' ',' ',' ',' ' ; Initialize with spaces for main grid
    num_board db '1','2','3','4','5','6','7','8','9' ; Separate number grid for input
positions

    ; Messages and prompts with proper line breaks
    title_msg db "Assembly Tic Tac Toe", 0Dh, 0Ah, "$"
    subtitle_msg db "Made by AAA (Aryan, Ajwad and Amreen)", 0Dh, 0Ah, 0Dh,
0Ah, "$"
    press_space db "Press SPACE to continue...", 0Dh, 0Ah, "$"

    p1_name_prompt db "Enter Player 1 name: $"
    p2_name_prompt db 0Dh, 0Ah, "Enter Player 2 name: $"

    timer_prompt db 0Dh, 0Ah, "Choose timer per move: 1=10s, 2=20s, 3=40s: $"

    toss_msg db 0Dh, 0Ah, "Coin toss... Press any key for result.", 0Dh, 0Ah, "$"
    x_starts db " wins the toss and starts as X!", 0Dh, 0Ah, "$"

    player_info db " (X) vs ", "$"
    player_info2 db " (O)", 0Dh, 0Ah, "$"

    p_turn_msg db "'s turn ($). Position (1-9): $"
    invalid_msg db 0Dh, 0Ah, "Invalid move! Try again: $"
    win_msg db " WINS! Congratulations!", 0Dh, 0Ah, "$"
    draw_msg db "IT'S A DRAW! Good game both players!", 0Dh, 0Ah, "$"
    timer_msg db "Time left: ?? seconds $"
    times_up db 0Dh, 0Ah, "Time's up! Next player's turn.", 0Dh, 0Ah, "$"
```

```
    replay_msg db 0Dh, 0Ah, "Press R for rematch or Q to quit: $"

    horz_line db "---+---+---        ---+---+---", 0Dh, 0Ah, "$"
    history_msg db 0Dh, 0Ah, "MOVE HISTORY:", 0Dh, 0Ah, "============",
0Dh, 0Ah, "$"
    view_history db 0Dh, 0Ah, "Press H to view history, any other key to
continue.", 0Dh, 0Ah, "$"

    ; Score display messages
    score_header db 0Dh, 0Ah, "CURRENT SCORES:", 0Dh, 0Ah,
"==============", 0Dh, 0Ah, "$"
    score_sep db ": $"
    newline db 0Dh, 0Ah, "$"

    ; Additional messages for better game experience
    game_result_header db 0Dh, 0Ah, "GAME OVER!", 0Dh, 0Ah, "=========",
0Dh, 0Ah, "$"

    ; Simple history storage - using simpler approach
    history_players db 9 dup(0)  ; Which player made each move (1 or 2)
    history_marks db 9 dup(0)    ; What mark they used (X or O)
    history_positions db 9 dup(0) ; What position they chose (1-9)

    ; Game state variables
    current_player db 1 ; 1 for Player 1, 2 for Player 2
    game_over db 0 ; 0 = ongoing, 1 = game over
    p1_score db 0 ; Player 1 score
    p2_score db 0 ; Player 2 score
    timer_limit db 10 ; Default 10 seconds per move
    timer_count db 10 ; Current timer
    current_mark db 'X' ; Current player's mark (X or O)
    winner_player db 0 ; 0 = draw, 1 = player 1, 2 = player 2
    p1_name db 20 dup('$') ; Player 1 name buffer
    p2_name db 20 dup('$') ; Player 2 name buffer
    toss_winner db 1 ; Who starts as X (1 or 2)
    history_count db 0 ; Number of moves recorded
    last_position db 0 ; Store last position played
```

```
.code
main proc
    mov ax, @data
    mov ds, ax

start_game:
    call title_screen
    call get_player_names
    call coin_toss
    call reset_game

game_loop:
    call clear_screen
    call display_score
    call display_boards
    call check_game_over
    cmp [game_over], 1
    je game_end
    call player_turn
    jmp game_loop

game_end:
    call display_result
    call ask_replay
    cmp al, 1 ; 1 means restart
    je start_game

    mov ah, 4Ch
    int 21h
main endp

;
==================================================================
====
; ARYAN'S SECTION - Features 1 & 2: Player turns and Coin toss
```

```
;
==========================================================
====

; Show title screen
title_screen proc
    call clear_screen
    mov ah, 9
    mov dx, offset title_msg
    int 21h
    mov dx, offset subtitle_msg
    int 21h
    mov dx, offset press_space
    int 21h

wait_space:
    mov ah, 0
    int 16h
    cmp al, ' '
    jne wait_space
    ret
title_screen endp

; Get player names and timer choice
get_player_names proc
    call clear_screen

    ; Get Player 1 name
    mov ah, 9
    mov dx, offset p1_name_prompt
    int 21h

    mov ah, 0Ah
    mov dx, offset p1_name
    mov byte ptr [p1_name], 18 ; Max length
    int 21h
```

```
    ; Null-terminate name
    mov si, offset p1_name + 2
    mov cl, [p1_name + 1]
    mov ch, 0
    add si, cx
    mov byte ptr [si], '$'

    ; Get Player 2 name
    mov ah, 9
    mov dx, offset p2_name_prompt
    int 21h

    mov ah, 0Ah
    mov dx, offset p2_name
    mov byte ptr [p2_name], 18
    int 21h

    ; Null-terminate name
    mov si, offset p2_name + 2
    mov cl, [p2_name + 1]
    mov ch, 0
    add si, cx
    mov byte ptr [si], '$'

    ; Get timer choice
timer_input:
    mov ah, 9
    mov dx, offset timer_prompt
    int 21h

    mov ah, 1
    int 21h
    sub al, '0'

    cmp al, 1
    je set_10
    cmp al, 2
```

```
    je set_20
    cmp al, 3
    je set_40
    jmp timer_input ; Invalid, retry

set_10:
    mov [timer_limit], 10
    jmp done_timer
set_20:
    mov [timer_limit], 20
    jmp done_timer
set_40:
    mov [timer_limit], 40
done_timer:
    ret
get_player_names endp

; Coin toss to decide who starts as X
coin_toss proc
    mov ah, 9
    mov dx, offset toss_msg
    int 21h

    mov ah, 0
    int 16h ; Wait for key

    ; Simple random: Use BIOS time
    mov ah, 0
    int 1Ah ; Get tick count in DX
    mov ax, dx
    and ax, 1 ; 0 or 1
    inc ax ; 1 or 2
    mov [toss_winner], al

    ; Display result with proper formatting
    mov ah, 9
    mov dx, offset newline
```

```
    int 21h

    ; Print winner name
    cmp al, 1
    je toss_p1
    mov dx, offset p2_name + 2
    jmp print_toss
toss_p1:
    mov dx, offset p1_name + 2
print_toss:
    mov ah, 9
    int 21h
    mov dx, offset x_starts
    int 21h

    ; Delay to show result
    mov cx, 1Eh
    mov dx, 8480h
    mov ah, 86h
    int 15h
    ret
coin_toss endp

; Handle player's turn - FIXED to show correct player and mark
player_turn proc
    ; Display whose turn with clear indication
    mov ah, 9
    mov dx, offset newline
    int 21h

    ; Show current player name
    cmp [current_player], 1
    je p1_turn_display
    mov dx, offset p2_name + 2
    jmp show_turn_name
p1_turn_display:
    mov dx, offset p1_name + 2
```

```
show_turn_name:
    mov ah, 9
    int 21h

    ; Update turn message with current mark
    mov al, [current_mark]
    mov [p_turn_msg + 9], al

    mov dx, offset p_turn_msg
    int 21h

    ; Initialize timer
    mov al, [timer_limit]
    mov [timer_count], al

timer_loop:
    ; Display timer at same position (no line breaks)
    mov ah, 2
    mov bh, 0
    mov dh, 15  ; Fixed row
    mov dl, 0   ; Fixed column
    int 10h

    ; Update timer message (handle 2 digits properly)
    mov ax, 0
    mov al, [timer_count]
    cmp al, 10
    jl single_digit_timer

    ; Two digit timer
    mov bl, 10
    div bl
    add al, '0'
    add ah, '0'
    mov [timer_msg + 11], al
    mov [timer_msg + 12], ah
    jmp show_timer
```

```
single_digit_timer:
    mov [timer_msg + 11], '0'
    add al, '0'
    mov [timer_msg + 12], al

show_timer:
    mov ah, 9
    mov dx, offset timer_msg
    int 21h

    ; Check input non-blocking
    mov ah, 1
    int 16h
    jnz get_input

    ; Delay ~1s
    mov cx, 0Fh
    mov dx, 4240h
    mov ah, 86h
    int 15h

    dec [timer_count]
    jz time_expired
    jmp timer_loop

time_expired:
    mov ah, 9
    mov dx, offset times_up
    int 21h
    ; Delay to show message
    mov cx, 1Eh
    mov dx, 8480h
    mov ah, 86h
    int 15h
    call switch_player
    ret
```

```
get_input:
    mov ah, 0
    int 16h
    sub al, '0'

    cmp al, 1
    jl invalid_input
    cmp al, 9
    jg invalid_input

    mov bx, 0
    mov bl, al
    dec bl
    mov cl, [board + bx]
    cmp cl, ' '
    jne invalid_input ; Already taken

    ; Mark position
    mov cl, [current_mark]
    mov [board + bx], cl

    ; Store position for history
    mov [last_position], al

    ; Record history
    call record_move

    call switch_player
    ret

invalid_input:
    mov ah, 9
    mov dx, offset invalid_msg
    int 21h
    mov ah, 1
    int 21h
```

```
    jmp get_input
player_turn endp

; Switch player - FIXED to handle marks correctly
switch_player proc
    ; Switch the current player number
    cmp [current_player], 1
    je set_p2
    mov [current_player], 1
    jmp set_mark
set_p2:
    mov [current_player], 2

set_mark:
    ; Set the correct mark based on who is X and who is O
    ; The toss winner is always X
    mov al, [current_player]
    cmp al, [toss_winner]
    je set_x_mark
    mov [current_mark], 'O'
    ret
set_x_mark:
    mov [current_mark], 'X'
    ret
switch_player endp

;
==============================================================
====
; AJWAD'S SECTION - Features 3 & 4: Winner checking and Draw detection
;
==============================================================
====

; Check if the game is over (win or draw)
check_game_over proc
    ; Check rows
```

```
    mov cx, 3
    mov si, 0
check_rows:
    mov al, [board+si]
    cmp al, ' '
    je next_row
    cmp al, [board+si+1]
    jne next_row
    cmp al, [board+si+2]
    jne next_row
    ; We have a winner
    call set_winner
    ret
next_row:
    add si, 3
    loop check_rows

    ; Check columns
    mov cx, 3
    mov si, 0
check_cols:
    mov al, [board+si]
    cmp al, ' '
    je next_col
    cmp al, [board+si+3]
    jne next_col
    cmp al, [board+si+6]
    jne next_col
    ; We have a winner
    call set_winner
    ret
next_col:
    inc si
    loop check_cols

    ; Check diagonals
    mov al, [board+0]
```

```
    cmp al, ' '
    je check_diag2
    cmp al, [board+4]
    jne check_diag2
    cmp al, [board+8]
    jne check_diag2
    ; We have a winner
    call set_winner
    ret

check_diag2:
    mov al, [board+2]
    cmp al, ' '
    je check_draw
    cmp al, [board+4]
    jne check_draw
    cmp al, [board+6]
    jne check_draw
    ; We have a winner
    call set_winner
    ret

check_draw:
    ; Check if board is full (draw)
    mov cx, 9
    mov si, 0
check_full:
    cmp [board+si], ' '
    je not_full
    inc si
    loop check_full

    ; Board is full, it's a draw
    mov [game_over], 1
    mov [winner_player], 0
    ret
```

```
not_full:
    ret
check_game_over endp

; Set winner based on current player who just moved - FIXED
set_winner proc
    mov [game_over], 1
    ; The winner is the current player (who just made the winning move)
    mov al, [current_player]
    mov [winner_player], al
    ret
set_winner endp

; Display the game result with enhanced messages
display_result proc
    call clear_screen
    call display_boards

    ; Show game over header
    mov ah, 9
    mov dx, offset game_result_header
    int 21h

    ; Check if it's a draw first
    cmp [winner_player], 0
    je show_draw

    ; It's a win - show winner message with celebration
    mov ah, 2
    mov dl, '*'
    int 21h
    mov dl, '*'
    int 21h
    mov dl, '*'
    int 21h
    mov dl, ' '
    int 21h
```

```
    cmp [winner_player], 1
    je p1_won

    ; Player 2 won
    mov ah, 9
    mov dx, offset p2_name + 2
    int 21h
    mov dx, offset win_msg
    int 21h
    inc [p2_score]
    jmp show_celebration

p1_won:
    mov ah, 9
    mov dx, offset p1_name + 2
    int 21h
    mov dx, offset win_msg
    int 21h
    inc [p1_score]
    jmp show_celebration

show_draw:
    ; Draw message with border
    mov ah, 2
    mov dl, '='
    int 21h
    mov dl, '='
    int 21h
    mov dl, '='
    int 21h
    mov dl, ' '
    int 21h
    mov ah, 9
    mov dx, offset draw_msg
    int 21h
    jmp result_done
```

```
show_celebration:
    ; Add celebration stars only for wins
    mov ah, 2
    mov dl, ' '
    int 21h
    mov dl, '*'
    int 21h
    mov dl, '*'
    int 21h
    mov dl, '*'
    int 21h
    mov ah, 9
    mov dx, offset newline
    int 21h

result_done:
    call display_score

    ; Show updated history
    mov ah, 9
    mov dx, offset newline
    int 21h
    mov dx, offset view_history
    int 21h

    mov ah, 0
    int 16h
    cmp al, 'H'
    je show_final_history
    cmp al, 'h'
    je show_final_history
    ret

show_final_history:
    call display_history
    mov ah, 0
```

```
    int 16h
    ret
display_result endp

; Ask if players want to play again
ask_replay proc
    mov ah, 9
    mov dx, offset replay_msg
    int 21h

wait_key:
    mov ah, 0
    int 16h
    cmp al, 'R'
    je replay_yes
    cmp al, 'r'
    je replay_yes
    cmp al, 'Q'
    je quit_game
    cmp al, 'q'
    je quit_game
    jmp wait_key

replay_yes:
    mov al, 1 ; Return 1 for restart
    ret

quit_game:
    ; Show final goodbye message
    call clear_screen
    mov ah, 2
    mov dl, 'T'
    int 21h
    mov dl, 'h'
    int 21h
    mov dl, 'a'
    int 21h
```

```
mov dl, 'n'
int 21h
mov dl, 'k'
int 21h
mov dl, 's'
int 21h
mov dl, ' '
int 21h
mov dl, 'f'
int 21h
mov dl, 'o'
int 21h
mov dl, 'r'
int 21h
mov dl, ' '
int 21h
mov dl, 'p'
int 21h
mov dl, 'l'
int 21h
mov dl, 'a'
int 21h
mov dl, 'y'
int 21h
mov dl, 'i'
int 21h
mov dl, 'n'
int 21h
mov dl, 'g'
int 21h
mov dl, '!'
int 21h
mov ah, 9
mov dx, offset newline
int 21h
mov al, 0 ; Return 0 for quit
ret
```

```
ask_replay endp

; Display both number grid and main grid side by side with better alignment
display_boards proc
    ; Display current player assignments more clearly
    mov ah, 9
    mov dx, offset newline
    int 21h

    ; Show who is X and who is O based on toss winner
    cmp [toss_winner], 1
    je p1_is_x

    ; P2 is X, P1 is O
    mov dx, offset p2_name + 2
    mov ah, 9
    int 21h
    mov dx, offset player_info
    int 21h
    mov dx, offset p1_name + 2
    int 21h
    mov dx, offset player_info2
    int 21h
    jmp show_grids

p1_is_x:
    ; P1 is X, P2 is O
    mov dx, offset p1_name + 2
    mov ah, 9
    int 21h
    mov dx, offset player_info
    int 21h
    mov dx, offset p2_name + 2
    int 21h
    mov dx, offset player_info2
    int 21h
```

```
show_grids:
    mov ah, 9
    mov dx, offset newline
    int 21h

    ; Add grid headers with better spacing
    mov ah, 2
    mov dl, ' '
    int 21h
    mov dl, ' '
    int 21h
    mov dl, 'P'
    int 21h
    mov dl, 'o'
    int 21h
    mov dl, 's'
    int 21h
    mov dl, 'i'
    int 21h
    mov dl, 't'
    int 21h
    mov dl, 'i'
    int 21h
    mov dl, 'o'
    int 21h
    mov dl, 'n'
    int 21h
    mov dl, 's'
    int 21h
    mov dl, ' '
    int 21h
    mov dl, ' '
    int 21h
    mov dl, ' '
    int 21h
    mov dl, ' '
    int 21h
```

```
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, 'G'
int 21h
mov dl, 'a'
int 21h
mov dl, 'm'
int 21h
mov dl, 'e'
int 21h
mov dl, ' '
int 21h
mov dl, 'B'
int 21h
mov dl, 'o'
int 21h
mov dl, 'a'
int 21h
mov dl, 'r'
int 21h
mov dl, 'd'
int 21h

mov ah, 9
mov dx, offset newline
int 21h

; Display grids side by side - Row 1 with better alignment
mov ah, 2
```

```
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+0]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+1]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+2]
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
```

```
int 21h
mov dl, ' '
int 21h
mov dl, [board+0]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [board+1]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [board+2]
int 21h
mov dl, ' '
int 21h

mov ah, 9
mov dx, offset newline
int 21h
mov dx, offset horz_line
int 21h

; Display grids side by side - Row 2
mov ah, 2
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+3]
```

```
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+4]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+5]
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, [board+3]
int 21h
```

```
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [board+4]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [board+5]
int 21h
mov dl, ' '
int 21h

mov ah, 9
mov dx, offset newline
int 21h
mov dx, offset horz_line
int 21h

; Display grids side by side - Row 3
mov ah, 2
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+6]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
```

```
mov dl, ' '
int 21h
mov dl, [num_board+7]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
int 21h
mov dl, [num_board+8]
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, ' '
int 21h
mov dl, [board+6]
int 21h
mov dl, ' '
int 21h
mov dl, '|'
int 21h
mov dl, ' '
```

```
    int 21h
    mov dl, [board+7]
    int 21h
    mov dl, ' '
    int 21h
    mov dl, '|'
    int 21h
    mov dl, ' '
    int 21h
    mov dl, [board+8]
    int 21h
    mov dl, ' '
    int 21h

    ; Option to view history
    mov ah, 9
    mov dx, offset view_history
    int 21h

    mov ah, 0
    int 16h
    cmp al, 'H'
    je show_history
    cmp al, 'h'
    je show_history
    ret

show_history:
    call display_history
    ; Wait for key to continue
    mov ah, 0
    int 16h
    ret
display_boards endp
```

```
;
==============================================================
====
; AMREEN'S SECTION - Features 5 & 6: Move validation and History/Scores
;
==============================================================
====

; Initialize or reset the game (keep scores)
reset_game proc
    ; Initialize main board with spaces
    mov cx, 9
    mov si, 0
init_board:
    mov [board+si], ' '
    inc si
    loop init_board

    ; Reset game state
    mov al, [toss_winner]
    mov [current_player], al
    mov [game_over], 0
    mov [current_mark], 'X'
    mov [winner_player], 0
    mov [history_count], 0

    ; Clear history arrays
    mov cx, 9
    mov si, 0
clear_history:
    mov [history_players+si], 0
    mov [history_marks+si], 0
    mov [history_positions+si], 0
    inc si
    loop clear_history

    ; Set timer
```

```
    mov al, [timer_limit]
    mov [timer_count], al
    ret
reset_game endp

; Record move in history - simplified approach
record_move proc
    mov al, [history_count]
    mov ah, 0
    mov si, ax

    ; Record which player made the move
    mov al, [current_player]
    mov [history_players+si], al

    ; Record what mark they used
    mov al, [current_mark]
    mov [history_marks+si], al

    ; Record position (convert to ASCII)
    mov al, [last_position]
    add al, '0'
    mov [history_positions+si], al

    inc [history_count]
    ret
record_move endp

; Display current score with proper formatting
display_score proc
    mov ah, 9
    mov dx, offset score_header
    int 21h

    ; Display Player 1: Score
    mov dx, offset p1_name + 2
    int 21h
```

```
    mov dx, offset score_sep
    int 21h
    mov ah, 2
    mov dl, [p1_score]
    add dl, '0'
    int 21h
    mov ah, 9
    mov dx, offset newline
    int 21h

    ; Display Player 2: Score
    mov dx, offset p2_name + 2
    int 21h
    mov dx, offset score_sep
    int 21h
    mov ah, 2
    mov dl, [p2_score]
    add dl, '0'
    int 21h
    mov ah, 9
    mov dx, offset newline
    int 21h

    ret
display_score endp

; Display move history - simplified version
display_history proc
    call clear_screen
    mov ah, 9
    mov dx, offset history_msg
    int 21h

    mov cl, [history_count]
    cmp cl, 0
    je no_history
```

```
    mov ch, 0
    mov si, 0
    mov bl, 1 ; Move counter

history_loop:
    ; Display "Move X: "
    mov ah, 2
    mov dl, 'M'
    int 21h
    mov dl, 'o'
    int 21h
    mov dl, 'v'
    int 21h
    mov dl, 'e'
    int 21h
    mov dl, ' '
    int 21h
    mov dl, bl
    add dl, '0'
    int 21h
    mov dl, ':'
    int 21h
    mov dl, ' '
    int 21h

    ; Display player name
    mov al, [history_players+si]
    cmp al, 1
    je hist_p1
    mov ah, 9
    mov dx, offset p2_name + 2
    jmp show_move_info
hist_p1:
    mov ah, 9
    mov dx, offset p1_name + 2

show_move_info:
```

```
int 21h

; Display mark
mov ah, 2
mov dl, ' '
int 21h
mov dl, '('
int 21h
mov dl, [history_marks+si]
int 21h
mov dl, ')'
int 21h
mov dl, ' '
int 21h
mov dl, '-'
int 21h
mov dl, ' '
int 21h
mov dl, 'P'
int 21h
mov dl, 'o'
int 21h
mov dl, 's'
int 21h
mov dl, ':'
int 21h
mov dl, ' '
int 21h
mov dl, [history_positions+si]
int 21h

mov ah, 9
mov dx, offset newline
int 21h

inc si
inc bl
```

```
    loop history_loop
    jmp history_done

no_history:
    mov ah, 2
    mov dl, 'N'
    int 21h
    mov dl, 'o'
    int 21h
    mov dl, ' '
    int 21h
    mov dl, 'm'
    int 21h
    mov dl, 'o'
    int 21h
    mov dl, 'v'
    int 21h
    mov dl, 'e'
    int 21h
    mov dl, 's'
    int 21h
    mov dl, ' '
    int 21h
    mov dl, 'y'
    int 21h
    mov dl, 'e'
    int 21h
    mov dl, 't'
    int 21h
    mov dl, '.'
    int 21h

history_done:
    mov ah, 9
    mov dx, offset newline
    int 21h
    mov dx, offset press_space
```

```
    int 21h
    ret
display_history endp




; Clear the screen
clear_screen proc
    mov ah, 0
    mov al, 3 ; Text mode 80x25
    int 10h
    ret
clear_screen endp

end main
```